



Data Parallelism vs Model Parallelism

Scaling Deep Learning Training Across Multiple GPUs

Why Parallelism Matters in AI

The Challenge

Modern deep learning models exceed **single GPU memory & compute** capabilities, requiring distributed training strategies

The Solution

Parallelism strategies unlock scaling across many GPUs, enabling training of increasingly complex models

Two Fundamental Approaches

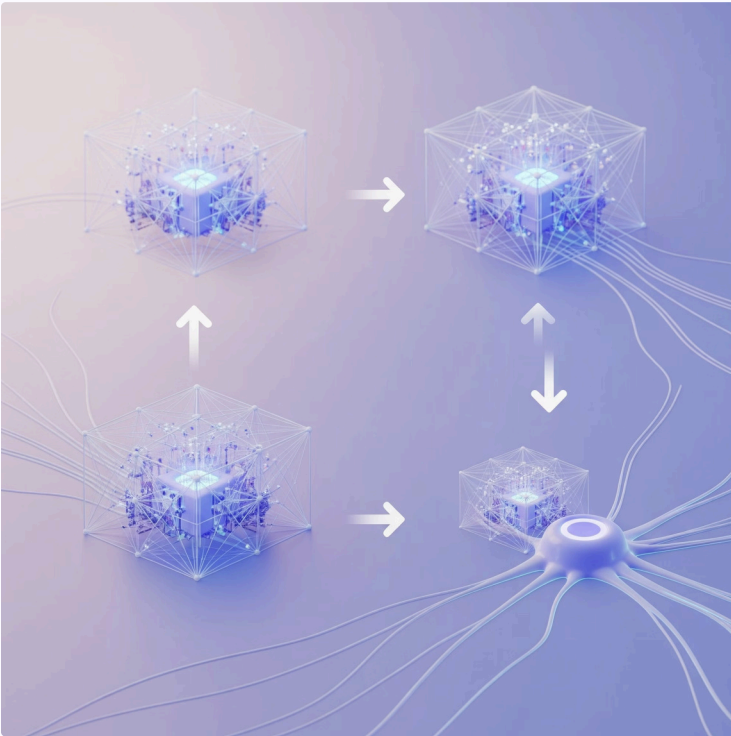
- **Data Parallelism (DP)**
- **Model Parallelism (MP)**

Your choice depends on:

- Model size
- Dataset size
- Infrastructure limitations

Many modern systems use **hybrid approaches** combining both strategies.

What Is Data Parallelism?



Data Parallelism splits your **dataset across multiple GPUs** while maintaining:

- Complete model copy on each GPU
- Different mini-batches processed in parallel
- Synchronized gradient updates via AllReduce

i Most common approach in frameworks like PyTorch DDP and TensorFlow MirroredStrategy

Data Parallelism Workflow

Split & Distribute

Divide mini-batch into N smaller batches and send to N different GPUs

Parallel Computation

Each GPU performs forward and backward pass independently with its portion of data

Gradient Synchronization

AllReduce operation aggregates gradients across all GPUs

Model Update

Update global model weights using synchronized gradients

Repeat

Continue with next batch until epoch completes

✓ Scales efficiently for medium/large datasets with moderately-sized models

Strengths of Data Parallelism



Implementation Simplicity

Modern frameworks provide high-level APIs requiring minimal code changes



Linear Scaling

Scales effectively to many GPUs with large batch sizes, often with near-linear speedup



Model Compatibility

No model restructuring required - works with existing architectures without modification



Hardware Optimization

Works well with specialized hardware like NCCL + NVLink clusters for efficient communication



Weaknesses of Data Parallelism

Memory Constraints

Entire model must fit in single GPU memory, limiting maximum model size

Communication Bottleneck

Gradient synchronization creates network overhead that grows with model size

Batch Size Challenges

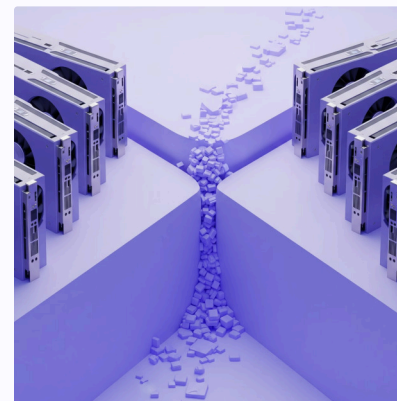
Larger effective batch sizes may harm model generalization without careful tuning

Straggler Problem

The slowest GPU determines overall training speed - system only as fast as weakest link

Diminishing Returns

Communication overhead leads to efficiency drops at extreme scale (1000+ GPUs)



What Is Model Parallelism?

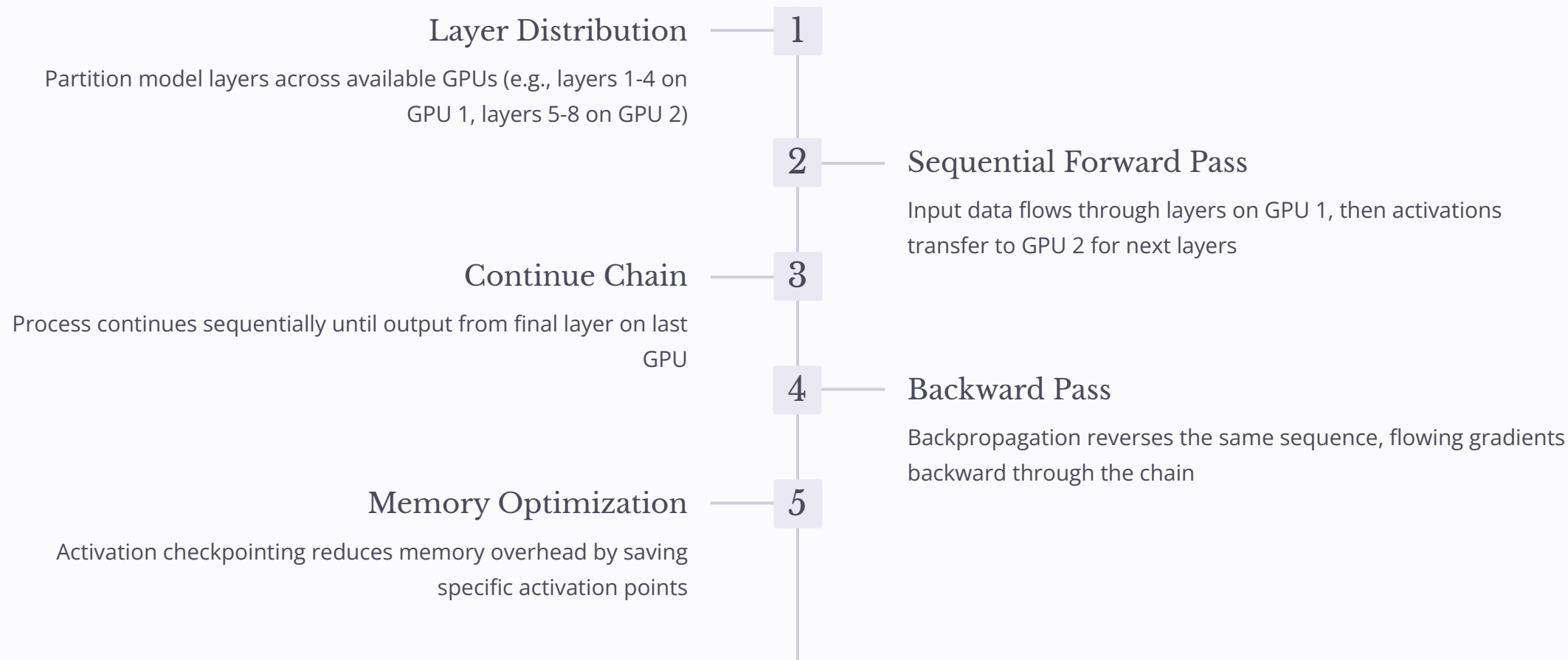
Instead of replicating the model, **Model Parallelism splits the model itself across multiple GPUs**:

- Each GPU holds specific layers or components
- Forward/backward passes flow sequentially across devices
- Required when **model size > single GPU memory**

❏ This approach is critical for training modern Large Language Models like GPT, PaLM, and LLaMA with billions or trillions of parameters



Model Parallelism Workflow



✓ Enables training of trillion-parameter models impossible with data parallelism alone



Strengths of Model Parallelism

Enables Ultra-Large Models

Breaks through single-GPU memory barriers, critical for modern LLMs with billions of parameters

Memory Efficiency

Each GPU only needs to hold its assigned portion of the model, reducing per-device memory requirements

Pipeline Integration

Can be combined with pipeline parallelism for even more efficient training of large models

Advanced Techniques

Supports memory optimization strategies like ZeRO (Zero Redundancy Optimizer) and FSDP (Fully Sharded Data Parallel)

Weaknesses of Model Parallelism

Implementation Complexity

Significantly more difficult to implement and debug than data parallelism

Partitioning Challenges

Requires careful model analysis to determine optimal layer distribution across GPUs

Communication Overhead

Frequent transfer of activations between devices can create bandwidth bottlenecks

Load Balancing

Uneven workload distribution leads to idle GPUs waiting for busier ones to complete

Training Time Increase

Sequential nature typically results in longer training times without careful optimization



Key Takeaways

Data Parallelism

- Replicate model, split data across GPUs
- Simple implementation, efficient scaling
- Limited by single GPU memory capacity
- Best for medium-sized models with large datasets

Model Parallelism

- Split model across GPUs, process sequentially
- Complex implementation, powerful capability
- Overcomes GPU memory limitations
- Essential for billion+ parameter models



Modern Best Practice

Most state-of-the-art AI systems use **hybrid approaches** combining Data Parallelism, Model Parallelism, and Pipeline Parallelism to maximize efficiency

- ✅ The right parallelism strategy depends on your specific model architecture, dataset characteristics, and available hardware infrastructure